

DSA + OOPS + clean code

Project

→ Console based.

→ Extendable.

↳ FE + API
based

Uber Ride Booking
Platform

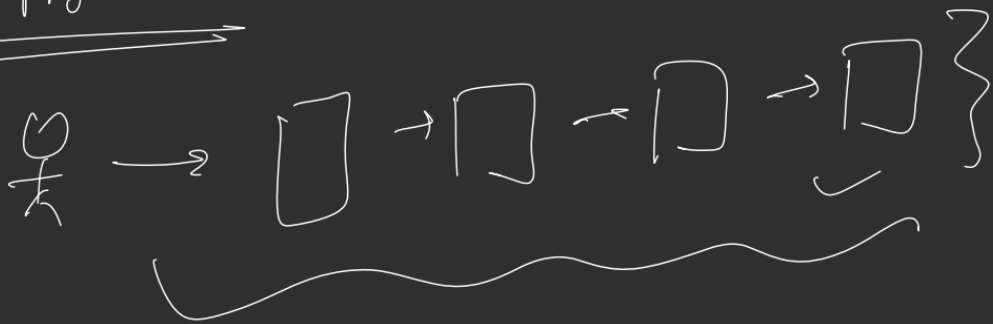
→ Requirements → (Algorithms)
→ Clean Code (Low-level design)
→ Code.

Uber Ride booking Platform

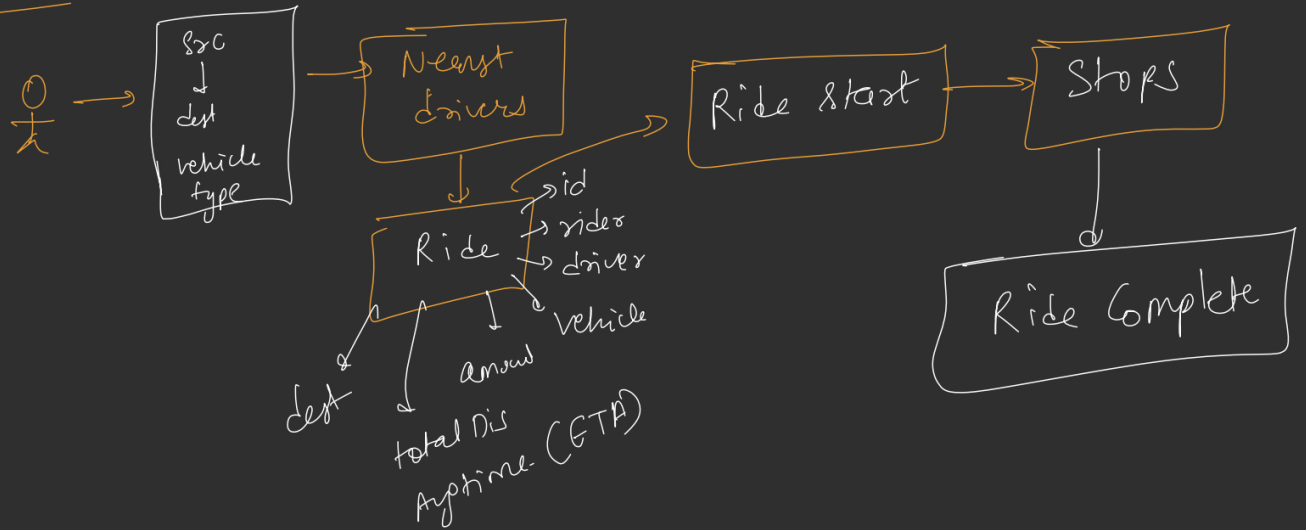
Requirement

- ① Rider can book a ride.
(Car/Auto/Bike/Suva) ---
- ② Nearest driver → Notification
- ③ Ride start
- ④ Src → dest (shortest path).
- ⑤ Rider → stop
- ⑥ Rider / Driver → Cancellation
- ⑦ Payment strategy → Pool + Surge +
ride charge
- ⑧ Cancellation
fee. Normal
charge

② Happy Flow



Happy Flow :



Algorithms (DSA)

Shortest dis b/w 2 points.

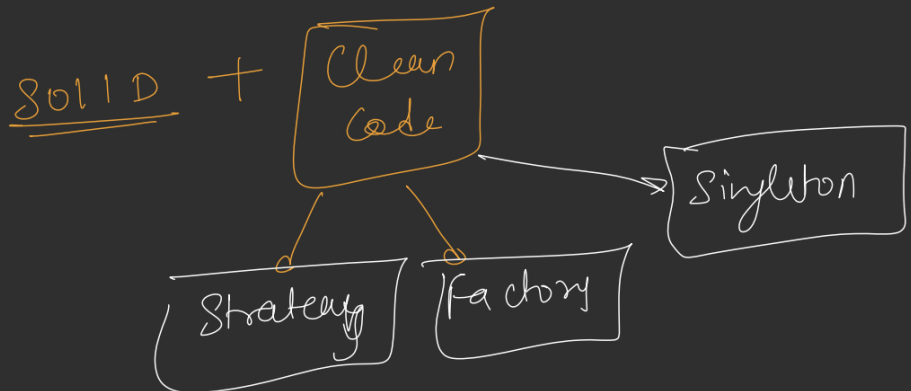
- ↳ Bellman Ford ??(x)
- ↳ Dijkstra Algo
- ↳ Floyd-W Algo
- ↳ (A*) Algo (✓)

Nearest K Drivers } PQ(✓)

Haversine Algo

Dynamic Pricing

UML Diagram



S : Single Responsibility Principle.

```
Class UserService {  
    BillCalculator bc;  
    getAllUsers()  
    removeUser()  
    printBill() {  
        bc → printBill();  
    }  
}  
  
Class BillCalculator {  
    printBill() {  
    }  
}  
  
Class Printer {  
    }  
}
```

○ : Open - Close principle.

```
Class A {  
    g1() { }  
    g2() { }  
    g3() { }  
}
```

```
class Persistence {  
    sqlPersistence (String data) {  
    }  
}
```

```
class persistence {  
    save();  
}
```

```
class sqlPersistence : persistence {  
    save() {  
    }  
}
```

```
class MongoP : persistence {  
    save() {  
    }  
}
```

2: Liskov Substitution Principle

